

Compilation

Compilation is the process of translating the source code that you write into a lower level language – either assembly (as is the case with Go), or some other intermediary language (as with Java and C#).

Compiled languages can be unpleasant to work with because compilation can be slow. It's hard to iterate quickly if you have to spend minutes or hours waiting for code to compile. Compilation speed is one of the major design goals of Go. This is good news for people working on large projects as well as those of us used to a quick feedback cycle offered by interpreted languages.

Compiled languages tend to run faster and the executable can be run without additional dependencies (at least, that's true for languages like C, C++ and Go which compile directly to assembly).

Static Typing

Being statically typed means that variables must be of a specific type (int, string, bool, []byte, etc.). This is either achieved by specifying the type when the variable is declared or, in many cases, letting the compiler infer the type (we'll look at examples shortly).

There's a lot more that can be said about static typing, but I believe it's something better understood by looking at code. If you're used to dynamically typed languages, you might find this cumbersome. You're not wrong, but there are advantages, especially when you pair static typing with compilation. The two are often conflated. It's true that when you have one, you normally have the other but it isn't a hard rule. With a rigid type system, a compiler is able to detect problems beyond mere syntactical mistakes as well as make further optimizations.

C-Like Syntax

Saying that a language has a C-like syntax means that if you're used to any other C-like languages such as C, C++, Java, JavaScript and C#, then you're going to find Go familiar – superficially, at least. For example, it means `&&` is used as a boolean AND, `==` is used to compare equality, `{` and `}` start and end a scope, and array indexes start at 0.

C-like syntax also tends to mean semi-colon terminated lines and parentheses around conditions. Go does away with both of these, though parentheses are still used to control precedence. For example, an `if` statement looks like this:

```
if name == "Leto" {  
    print("the spice must flow")  
}
```

And in more complicated cases, parentheses are still useful:

```
if (name == "Goku" && power > 9000) || (name == "gohan" && power < 4000) {  
    print("super Saiyan")  
}
```

Beyond this, Go is much closer to C than C# or Java - not only in terms of syntax, but in terms of purpose. That's reflected in the terseness and simplicity of the language which will hopefully start to become obvious as you learn it.

Garbage Collected

Some variables, when created, have an easy-to-define life. A variable local to a function, for example, disappears when the function exits. In other cases, it isn't so obvious – at least to a compiler. For example, the lifetime of a variable returned by a function or referenced by other variables and objects can be tricky to determine. Without garbage collection, it's up to developers to free the memory associated with such variables at a point where the developer knows the variable isn't needed. How? In C, you'd literally `free(str)`; the variable.

Languages with garbage collectors (e.g., Ruby, Python, Java, JavaScript, C#, Go) are able to keep track of these and free them when they're no longer used. Garbage collection adds overhead, but it also eliminates a number of devastating bugs.

Running Go Code

Let's start our journey by creating a simple program and learning how to compile and execute it. Open your favorite text editor and write the following code:

```
package main

func main() {
    println("it's over 9000!")
}
```

Save the file as `main.go`. For now, you can save it anywhere you want; we don't need to live inside Go's workspace for trivial examples.

Next, open a shell/command prompt and change the directory to where you saved the file. For me, that means typing `cd ~/code`.

Finally, run the program by entering:

```
go run main.go
```

If everything worked, you should see *it's over 9000!*.

But wait, what about the compilation step? `go run` is a handy command that compiles *and* runs your code. It uses a temporary directory to build the program, executes it and then cleans itself up. You can see the location of the temporary file by running:

```
go run --work main.go
```

To explicitly compile code, use `go build`:

```
go build main.go
```

This will generate an executable `main` which you can run. On Linux / OSX, don't forget that you need to prefix the executable with dot-slash, so you need to type `./main`.

While developing, you can use either `go run` or `go build`. When you deploy your code however, you'll want to deploy a binary via `go build` and execute that.

Main

Hopefully, the code that we just executed is understandable. We've created a function and printed out a string with the built-in `println` function. Did `go run` know what to execute because there was only a single choice? No. In Go, the entry point to a program has to be a function called `main` within a package `main`.

We'll talk more about packages in a later chapter. For now, while we focus on understanding the basics of Go, we'll always write our code within the `main` package.

If you want, you can alter the code and change the package name. Run the code via `go run` and you should get an error. Then, change the name back to `main` but use a different function name. You should see a different error

message. Try making those same changes but use `go build` instead. Notice that the code compiles, there's just no entry point to run it. This is perfectly normal when you are, for example, building a library.

Imports

Go has a number of built-in functions, such as `println`, which can be used without reference. We can't get very far though, without making use of Go's standard library and eventually using third-party libraries. In Go, the `import` keyword is used to declare the packages that are used by the code in the file.

Let's change our program:

```
package main

import (
    "fmt"
    "os"
)

func main() {
    if len(os.Args) != 2 {
        os.Exit(1)
    }
    fmt.Println("It's over", os.Args[1])
}
```

Which you can run via:

```
go run main.go 9000
```

We're now using two of Go's standard packages: `fmt` and `os`. We've also introduced another built-in function `len`. `len` returns the size of a string, or the number of values in a dictionary, or, as we see here, the number of elements in an array. If you're wondering why we expect 2 arguments, it's because the first argument – at index 0 – is always the path of the currently running executable. (Change the program to print it out and see for yourself.)

You've probably noticed we prefix the function name with the package, e.g., `fmt.Println`. This is different from many other languages. We'll learn more about packages in later chapters. For now, knowing how to import and use a package is a good start.

Go is strict about importing packages. It will not compile if you import a package but don't use it. Try to run the following:

```
package main

import (
    "fmt"
    "os"
)

func main() {
}
```

You should get two errors about `fmt` and `os` being imported and not used. Can this get annoying? Absolutely. Over time, you'll get used to it (it'll still be annoying though). Go is strict about this because unused imports can slow compilation; admittedly a problem most of us don't have to this degree.

Another thing to note is that Go's standard library is well documented. You can head over to <https://golang.org/pkg/fmt/#Println> to learn more about the `Println` function that we used. You can click on that section header and see the source code. Also, scroll to the top to learn more about Go's formatting capabilities.

If you're ever stuck without internet access, you can get the documentation running locally via:

```
godoc -http=:6060
```

and pointing your browser to `http://localhost:6060`

Variables and Declarations

It'd be nice to begin and end our look at variables by saying *you declare and assign to a variable by doing `x = 4`*. Unfortunately, things are more complicated in Go. We'll begin our conversation by looking at simple examples. Then, in the next chapter, we'll expand this when we look at creating and using structures. Still, it'll probably take some time before you truly feel comfortable with it.

You might be thinking *Woah! What can be so complicated about this?* Let's start looking at some examples.

The most explicit way to deal with variable declaration and assignment in Go is also the most verbose:

```
package main

import (
    "fmt"
)

func main() {
    var power int
    power = 9000
    fmt.Printf("It's over %d\n", power)
}
```

Here, we declare a variable `power` of type `int`. By default, Go assigns a zero value to variables. Integers are assigned 0, booleans `false`, strings `""` and so on. Next, we assign 9000 to our `power` variable. We can merge the first two lines:

```
var power int = 9000
```

Still, that's a lot of typing. Go has a handy short variable declaration operator, `:=`, which can infer the type:

```
power := 9000
```

This is handy, and it works just as well with functions:

```
func main() {
    power := getPower()
}

func getPower() int {
    return 9001
}
```

It's important that you remember that `:=` is used to declare the variable as well as assign a value to it. Why? Because a variable can't be declared twice (not in the same scope anyway). If you try to run the following, you'll get an error.

```
func main() {
    power := 9000
```

```

fmt.Printf("It's over %d\n", power)

// COMPILER ERROR:
// no new variables on left side of :=
power := 9001
fmt.Printf("It's also over %d\n", power)
}

```

The compiler will complain with *no new variables on left side of :=*. This means that when we first declare a variable, we use `:=` but on subsequent assignment, we use the assignment operator `=`. This makes a lot of sense, but it can be tricky for your muscle memory to remember when to switch between the two.

If you read the error message closely, you'll notice that *variables* is plural. That's because Go lets you assign multiple variables (using either `=` or `:=`):

```

func main() {
    name, power := "Goku", 9000
    fmt.Printf("%s's power is over %d\n", name, power)
}

```

As long as one of the variables is new, `:=` can be used. Consider:

```

func main() {
    power := 1000
    fmt.Printf("default power is %d\n", power)

    name, power := "Goku", 9000
    fmt.Printf("%s's power is over %d\n", name, power)
}

```

Although `power` is being used twice with `:=`, the compiler won't complain the second time we use it, it'll see that the other variable, `name`, is a new variable and allow `:=`. However, you can't change the type of `power`. It was declared (implicitly) as an integer and thus, can only be assigned integers.

For now, the last thing to know is that, like imports, Go won't let you have unused variables. For example,

```

func main() {
    name, power := "Goku", 1000
    fmt.Printf("default power is %d\n", power)
}

```

won't compile because `name` is declared but not used. Like unused imports it'll cause some frustration, but overall I think it helps with code cleanliness and readability.

There's more to learn about declaration and assignments. For now, remember that you'll use `var NAME TYPE` when declaring a variable to its zero value, `NAME := VALUE` when declaring and assigning a value, and `NAME = VALUE` when assigning to a previously declared variable.

Function Declarations

This is a good time to point out that functions can return multiple values. Let's look at three functions: one with no return value, one with one return value, and one with two return values.

```
func log(message string) {  
}  
  
func add(a int, b int) int {  
}  
  
func power(name string) (int, bool) {  
}
```

We'd use the last one like so:

```
value, exists := power("goku")  
if exists == false {  
    // handle this error case  
}
```

Sometimes, you only care about one of the return values. In these cases, you assign the other values to `_`:

```
_, exists := power("goku")  
if exists == false {  
    // handle this error case  
}
```

This is more than a convention. `_`, the blank identifier, is special in that the return value isn't actually assigned. This lets you use `_` over and over again regardless of the returned type.

Finally, there's something else that you're likely to run into with function declarations. If parameters share the same type, we can use a shorter syntax:

```
func add(a, b int) int {  
  
}
```

Being able to return multiple values is something you'll use often. You'll also frequently use `_` to discard a value. Named return values and the slightly less verbose parameter declaration aren't that common. Still, you'll run into all of these sooner than later so it's important to know about them.

Before You Continue

We looked at a number of small individual pieces and it probably feels disjointed at this point. We'll slowly build larger examples and hopefully, the pieces will start to come together.

If you're coming from a dynamic language, the complexity around types and declarations might seem like a step backwards. I don't disagree with you. For some systems, dynamic languages are categorically more productive.

If you're coming from a statically typed language, you're probably feeling comfortable with Go. Inferred types and multiple return values are nice (though certainly not exclusive to Go). Hopefully as we learn more, you'll appreciate the clean and terse syntax.